Ders 3 Genişletilmiş Notu: Öğrenci Sunumları: Süreçler Arası İletişim, REST ve gRPC, Saga Koordinasyonu, İş Mantığı Desenleri, Domain Event'ler ve Event Sourcing ile Sagalar

Ders 3 Genişletilmiş Notu: Öğrenci Sunumları: Süreçler Arası İletişim, REST ve gRPC, Saga Koordinasyonu, İş Mantığı Desenleri, Domain Event'ler ve Event Sourcing ile Sagalar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Süreçler arası iletişime genel bakış ve etkileşim stilleri
- 2. API tanımlama ve API-first tasarım
- 3. API'lerin evrimi ve anlamsal sürümleme
- 4. Mesaj formatları
- 5. Senkron uzak prosedür çağrısı ve REST
- 6. gRPC
- 7. Kısmi hata ve Circuit Breaker
- 8. Servis keşfi
- 9. Saga ile işlem koordinasyonu: neden gerekli
- 10. Koreografi tabanlı saga
- 11. Orkestrasyon tabanlı saga
- 12. Altıgen mimari ve iş mantığı yaklaşımları
- 13. Transaction Script deseni
- 14. Domain Model deseni ve Domain-Driven Design
- 15. Domain event'ler: tanım ve kullanım nedenleri
- 16. Event sınıfı, meta veri ve event zenginleştirme
- 17. Domain event'leri belirleme ve event storming
- 18. Domain event'leri üretme, güvenilir yayınlama ve tüketme
- 19. Örnek sistem: kitap ödünç alma
- 20. Saga türlerinin karşılaştırması ve event sourcing
- 21. Event sourcing ile koreografi sagaları
- 22. Event sourcing ile orkestrasyon sagaları ve katılımcıları

Bu ders altı öğrenci sunumundan oluşur. Önceki haftadan kalan üç sunum ile bu haftanın üç sunumu yapılır. Sunumlar dersin mikroservis kitabındaki bölümleri izler: birinci sunum süreçler arası iletişime genel bakışı (Bölüm 1-4), ikinci sunum senkron uzak prosedür çağrısını (Bölüm 5-8), üçüncü sunum saga ile işlem koordinasyonunu (Bölüm 9-11), dördüncü sunum iş mantığının düzenlenmesini (Bölüm 12-14), beşinci sunum domain event'lerin yayınlanmasını (Bölüm 15-19), altıncı sunum event sourcing ile sagaların birlikte kullanımını (Bölüm 20-22) anlatır. Sunan kişiler bu notta "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetmiş, akademik bir ek açıklama yapmamıştır.

Ek bilgi (derste anlatılmadı): Sunumlardaki şekiller ve örnekler (FTGO, `Order-`

`ServiceProxy`, Eventuate Tram, `SagaReplyRequested`) büyük olasılıkla Chris Richardson'ın *Microservices Patterns* kitabından alınmıştır. Bu kitapta süreçler arası iletişim ve RPI üçüncü bölümün, sagalar dördüncü bölümün, iş mantığı ve domain event'ler beşinci bölümün, event sourcing altıncı bölümün konusudur.

1. Süreçler arası iletişime genel bakış ve etkileşim stilleri

Birinci sunum İngilizce yapılır ve kitabın üçüncü bölümünün ilk kısmını, yani mikroservis mimarisinde süreçler arası iletişime (inter-process communication, IPC) genel bakışı anlatır. Bu sunumun altyazısının önemli bir kısmı Whisper tekrarlarıyla bozuktur; aşağıdaki anlatım okunabilen altyazı ve ekrandaki slaytlardan kurulmuştur.

1.1 Birinci ve ikinci bölümün tekrarı

Öğrenci önce kitabın ilk iki bölümünü hatırlatır (ekrandan, 00:10:00). Birinci bölümde monolitik mimari incelenmişti. Monolitik mimari basittir, küçük ekiplere ve küçük projelere uygundur, kısa sürede hızlı uygulama geliştirmeyi sağlar. Bunlar güçlü yönleridir; ama zayıf yönleri de çoktur. Örneğin monolit yalnız bir bütün olarak ölçeklenebilir. İkinci bölümde mikroservis mimarisi tanıtılmıştı. Öğrenciye göre mikroservis mimarisinin asıl gücü, servislerin bağımsız olarak dağıtılabilen (independently deployable) birimler olmasıdır.

1.2 Etkileşim stilleri

Etkileşim stili (interaction style), istemcilerin ve servislerin nasıl etkileştiğini teknolojiden bağımsız biçimde anlatma yoludur (ekrandan, 00:13:10). Önce etkileşim stilini düşünmek, IPC'nin ayrıntılarına takılıp kalmadan gereksinime odaklanmayı sağlar.

Etkileşim stilleri iki boyutta sınıflandırılır. Birinci boyut, bir isteği kaç servisin karşıladığıdır. Bire bir (one-to-one) etkileşimde her istemci isteğini tek bir servis karşılar. Bire çok (one-to-many) etkileşimde bir isteğin karşılanması için birden çok servis çalışabilir. İkinci boyut zamanla ilgilidir. Senkron etkileşimde istemci zamanında bir cevap bekler ve beklerken bloke bile olabilir. Asenkron etkileşimde istemci bloke olmaz; cevap varsa bile hemen gönderilmek zorunda değildir, istemci süreci bu arada başka işler yapabilir.

Slayttaki tablo bu iki boyutu birleştirir (ekrandan, 00:13:10):

	Bire bir	Bire çok
Senkron	Request/response	–
Asenkron	Asenkron request/response; tek yönlü bildirim (one-way notification)	Publish/subscribe; publish/async responses

Bire bir senkron etkileşimin örneği request/response'tur: servis istemcisi servise bir istek yapar ve cevabı bekler; istemci cevabın zamanında gelmesini bekler. Bire çok etkileşimin örneği publish/subscribe'dır: istemci bir bildirim mesajı yayınlar, bu mesajı sıfır ya da daha fazla ilgili servis tüketir.

2. API tanımlama ve API-first tasarım

2.1 Arayüzlerin rolü

Öğrenciye göre API'ler, yani arayüzler, her yazılım geliştirmenin merkezindedir. Bir uygulama, özellikle bu dersin ilgilendiği büyük ölçekli uygulamalar, çok sayıda modülden oluşur. Her modülün, istemcilerinin kullanabileceği işlemleri tanımlayan bir arayüzü olmalıdır. İyi tasarlanmış arayüzler işe yarar işlevselliği sunarken gerçekleştirmeyi (implementation) gizler. Böylece gerçekleştirme, istemcileri etkilemeden değişebilir. Bütün amaç servisleri ve istemcileri gevşek bağlı (loosely coupled) tutmaktır.

Bir arayüz genellikle bir programlama dili yapısıyla tanımlanır; Java'da bu Java arayüzleridir (interface). Gerçekleştirme sınıfları istemciden gizlidir. Öğrenci ardından Java'nın statik tipli

bir dil olduğunu ve arayüzlerde değişiklik olması durumunu ele alır; cümlenin devamı altyazıda bozuktur.

2.2 API-first tasarım

Mikroservis mimarisiyle yazılım geliştirirken API-first tasarım değerlendirilmesi gereken uygun bir seçenektir (ekrandan, 00:20:00). Adımları şöyledir:

- Önce arayüz tanımı yazılır.
- Sonra bu tanım istemci geliştiricileriyle birlikte gözden geçirilir.
- İstemci geliştiricileriyle birçok kez ileri geri gidilip bir arayüz üzerinde anlaşıldıktan sonra servis inşa edilir.

Yani önce servisi yazıp sonra API'yi düşünmek yerine süreç tersine çevrilir: önce API'ye odaklanılır ve istemcilerle anlaşmaya varılır, gerçek inşa buradan başlar.

Slayta göre API tanımının doğası seçilen IPC mekanizmasına bağlıdır. Mesajlaşma kullanılıyorsa API mesaj kanallarından, mesaj tiplerinden ve formatlarından oluşur. HTTP kullanılıyorsa API URL'lerden, HTTP fiillerinden ve istek ile cevap formatlarından oluşur.

3. API'lerin evrimi ve anlamsal sürümleme

3.1 Neden sorun

API'ler uygulama geliştikçe değişir. Öğrenciye göre mikroservis tabanlı bir uygulamada bir servisin istemcileri başka servisler ve başka ekipler olabilir. Ayrıca modern uygulamalar bakım için kapatılmaz, 7/24 çalışır. Bu yüzden eski ve yeni sürümün aynı anda çalıştığı kademeli güncelleme (rolling update) yapılır. Bu durum, API'lerin kontrollü biçimde evrilmesi için bir strateji gerektirir.

3.2 Anlamsal sürümleme

Anlamsal sürümleme (semantic versioning) genelde yazılım paketlerini sürümlemek için kullanılır, ama dağıtık bir sistemde API sürümlemesi için de kullanılabilir (ekrandan, 00:22:40). Üç parçadan oluşur: `MAJOR.MINOR.PATCH`.

- MAJOR: API'de uyumsuz bir değişiklik yapıldığında artırılır.
- MINOR: API'de geriye uyumlu bir iyileştirme yapıldığında artırılır.
- PATCH: geriye uyumlu bir hata düzeltmesi yapıldığında artırılır.

REST API gerçekleştiriliyorsa major sürüm URL yolunun ilk öğesi olarak kullanılabilir. Mesajlaşma kullanan bir servis gerçekleştiriliyorsa sürüm numarası servisin yayınladığı mesajlara konabilir. Amaç API'leri düzgün sürümlemek ve kontrollü biçimde evriltmektir.

3.3 Geriye uyumlu (minor) değişiklikler

Genel kural olarak her zaman yalnız geriye uyumlu değişiklik yapmaya çalışılmalıdır. Burada yol gösteren ilke sağlamlık ilkesidir (robustness principle): "Yaptığın işte tutucu ol, başkalarından kabul ettiğinde hoşgörülü ol." Buna göre servisler eksik istek alanları için varsayılan değerler sağlamalı, istemciler de cevapta fazladan gelen alanları yok saymalıdır. Bu doğru yapılırsa eski istemciler yeni servisle de çalışır. Geriye uyumlu değişikliğe örnek, isteğe ek alanlar eklemektir.

3.4 Major değişiklik örneği

Öğrencinin gösterdiği major değişiklik örneğinde istemci, bir siparişin 1.x sürümünü HTTP isteğinin başlığında belirterek ister (ekrandan, 00:24:40):

```
GET /orders/xyz HTTP/1.1
Accept: application/vnd.example.resource+json; version=1
...
```

Servis tarafında bu başlığı işleyip istenen sürüme göre gerekli hizmeti sağlayan bir mekanizma bulunmalıdır.

4. Mesaj formatları

IPC'nin özü mesaj alışverişidir. Mesajlar genellikle veri içerir, bazen içermeyebilir. Mesaj formatının seçimi önemli bir tasarım kararıdır, çünkü IPC'nin verimliliğini, API'nin kullanılabilirliğini ve evrilebilirliğini etkiler. Formatlar iki gruba ayrılır: metin tabanlı formatlar ve ikili (binary) formatlar.

4.1 Metin formatları

JSON ve XML metin tabanlı formatlardır. İnsan tarafından okunabilir ve kendini tanımlayan (self-describing) formatlardır. JSON adlandırılmış özelliklerin bir koleksiyonudur; XML de bir koleksiyondur ama adlandırılmış öğelerden ve değerlerden oluşur. Tüketici ihtiyaç duyduğu değerleri seçer, gerisini tamamen yok sayar.

4.2 İkili formatlar

Birçok ikili format vardır, ama popüler olanlar Protocol Buffers ve Avro'dur (ekrandan, 00:27:40). Protocol Buffers Google tarafından geliştirilmiştir, Avro ise bir Apache projesidir. Slayttaki karşılaştırma şöyledir:

- Protocol Buffers etiketli alanlar (tagged fields) kullanır.
- Avro tüketicisi ise mesajı yorumlayabilmek için şemayı bilmek zorundadır.
- Bu yüzden API evrimi Protocol Buffers ile Avro'ya göre daha kolaydır.
- İki format da mesajların yapısını tanımlamak için tipli bir IDL (interface definition language) sunar; bir derleyici mesajları serileştiren ve geri çözen kodu üretir.
- Bu durum servis tasarımında API-first yaklaşımını zorunlu kılar.
- Ayrıca istemci statik tipli bir dilde yazılırsa derleyici istemcinin API'yi doğru kullandığını denetler; öğrenciye göre bu ek bir kazançtır.

5. Senkron uzak prosedür çağrısı ve REST

İkinci sunum Türkçe yapılır ve birinci sunumun hemen devamı olan konuyu, yani uzak prosedür çağrısıyla (Remote Procedure Invocation, RPI) iletişimi anlatır. Öğrenci bağlamdan kopmamak için kitaba bağlı kaldığını söyler. Sunum önce REST'i, sonra gRPC'yi, ardından bu yöntemlerde yaşanan problemleri ve çözümlerini ele alır.

> **Ders sonrası not:** Öğrenci girişte "asenkron mesajlaşma desenini" anlatacağını söyledi; ama anlattığı RPI (REST, gRPC) senkron bir iletişim biçimidir. Ekrandaki kitap metni de gRPC için "REST gibi senkron bir iletişim mekanizmasıdır" der (ekrandan, 00:40:00).

5.1 RPI nedir

RPI'da bir istemci ve bir servis vardır. İkisinin iletişimi için bir vekil (proxy) ve bir arayüz bulunur. İstemci vekilin arayüzüne bir istekte bulunur, vekil servisle iletişime geçer ve cevabı istemciye geri getirir. Uzaktaki fonksiyonların ya da kaynakların çağrılması bu kapsama girer. RPI'ın çeşitli yöntemleri vardır: REST, gRPC, webhook ve WebSocket gibi.

5.2 REST'in temelleri

REST'i Roy Fielding tanımlamıştır. Fielding'in tanımına göre REST, ölçeklenebilir bileşen iletişimini sağlamak için geliştirilmiş, HTTP tabanlı bir iletişim stilidir. İstemci bir istek gönderir ve JSON ya da XML gibi insanın okuyabileceği bir formatta cevap alır. Kaynak (resource) denen şey bir temsil (representation) olarak görünür; istemci istek atarak bu temsile ulaşmak ister.

REST'in özellikleri şunlardır:

- İstemci-sunucu ayrımı nettir: istemcinin yaptığı iş ile sunucunun yaptığı iş birbirinden bağımsızdır. Bu da gevşek bağlılığı sağlar.
- Mesajlar insan okuyabilir metindir (JSON ya da XML). Buna karşılık gRPC ikili, insanın okuyamayacağı bir format kullanır.
- İletişim protokolü olarak HTTP/1.1 ve HTTP/2 desteklenir; GET ve POST gibi temel komutlar kullanılır.
- Durumsuzluk (stateless): sunucu verdiği cevabın önceki cevaplarla ilgisini bilmek zorunda değildir; her isteği ilk kez görüyormuş gibi cevaplar.
- Önbelleğe alınabilirlik (cacheable): bir isteğe gelen cevap önbelleğe konabilir ve tekrar gerektiğinde doğrudan önbellekten alınabilir. Bu hem hızı artırır hem trafiği azaltır.
- Tek biçimli arayüz (uniform interface).

5.3 REST olgunluk modeli

Öğrenciye göre geliştiricilerin çoğu HTTP tabanlı API'lerini RESTful diye tanımlar, ama bu her zaman doğru değildir. Bunu değerlendirmek için dört düzeyli bir REST olgunluk modeli vardır. Yalnız POST kullanılıyorsa düzey 0 ya da düzey 1'de kalınır. POST dışındaki HTTP fiilleri de uygun biçimde kullanılırsa daha etkili bir çalışma elde edilir ve düzey 2'ye çıkılır. Kod HATEOAS ilkesine uygun yazılırsa düzey 3 REST olgunluğuna erişilir.

> **Ders sonrası not:** Öğrenci bu modeli REST'i tanımlayan kişinin (Roy Fielding) geliştirdiğini söyledi. Bu model büyük olasılıkla Leonard Richardson'a aittir ve "Richardson Maturity Model" diye bilinir.
>
> **Ek bilgi (derste anlatılmadı):** Richardson modelinde düzey 0 her şeyi tek bir URL'ye POST ile gönderir; düzey 1 ayrı kaynaklar (URL'ler) tanımlar ama hâlâ tek fiil kullanır; düzey 2 HTTP fiillerini (GET, POST, PUT, DELETE) anlamlarına uygun kullanır; düzey 3'te (HATEOAS, Hypermedia as the Engine of Application State) cevaplar, istemcinin yapabileceği sonraki işlemlerin bağlantılarını içerir.

5.4 REST API'lerinin belirtimi

API'ler bir arayüz tanımlama diliyle (IDL) tanımlanmalıdır. Ekrandaki kitap metnine göre (ekrandan, 00:35:50), CORBA ve SOAP gibi eski iletişim protokollerinin aksine REST'in başta bir IDL'i yoktu. Neyse ki geliştirici topluluğu RESTful API'ler için IDL'in değerini yeniden keşfetti. En popüler REST IDL'i, Swagger açık kaynak projesinden evrilen Open API Specification'dır. Swagger, REST API geliştirmek ve belgelemek için bir araç takımıdır; bir arayüz tanımından istemci taslakları (stub) ve sunucu iskeletleri üretebilir.

> **Ders sonrası not:** Altyazıda öğrencinin "REST'te şans eseri IDL isimli bir arayüz tanımlama diliyle tanımlanabiliyor" gibi bir cümlesi geçer. Ekrandaki metne göre doğrusu: REST'in başlangıçta IDL'i yoktu; bugün en yaygın REST IDL'i OpenAPI Specification'dır.

5.5 REST'in sorunları, artıları ve eksileri

REST kullanırken iki temel sorun yaşanır. Birincisi, tek istekte birden çok kaynağa erişmek istendiğinde zorlanılır. Bunu çözmek için ya ikisi aynı anda getirilir ya da GraphQL ve Netflix'in Falcor'u gibi yeni teknolojiler kullanılır. İkincisi, REST'te dört HTTP fiiline bağlı kalmak gerekir ve bazı işlemler her zaman HTTP fiilleriyle ifade edilemez. Bunun önüne geçmek için Google gRPC'yi geliştirmiştir.

REST'in artıları: oldukça basit ve herkesin aşina olduğu bir yapıdır; HTTP fiillerini kullandığı için web geliştiricilerinin çoğu kolayca anlar; güvenlik duvarına (firewall) takılmaz, tehdit olarak algılanmaz. Eksileri: yalnız request/response stilini destekler; ayrıca yukarıdaki iki sorun (birden çok kaynağa erişim ve HTTP fiilleriyle ifade) vardır. Öğrenci bir şemada REST, gRPC gibi bütün yöntemleri RPI çemberi içinde gösterir: hepsi uzaktan çağrıyla fonksiyonlara ya da kaynaklara erişir.

6. gRPC

gRPC, REST'in az önceki dezavantajlarını aşmak için Google tarafından ortaya çıkarılmış bir sistemdir ve HTTP/2 protokolünü kullanır. Dilden bağımsız çalışır: Java, C#, Node.js, Go gibi dillerde ikili mesajlar istenen formata çevrilir. Mobilden ve masaüstünden, platformdan bağımsız biçimde çalıştırılabilir. Mesaj formatı olarak birinci sunumda anlatılan Protocol Buffers'ı kullanır. Oldukça verimli ve performanslıdır.

Ekrandaki kitap metnine göre gRPC'nin artıları ve eksileri şunlardır (ekrandan, 00:40:00):

- Artılar:
  - Zengin bir güncelleme işlemleri kümesine sahip bir API tasarlamak kolaydır. REST'te yalnız dört HTTP fiiline bağlı kalınıyordu.
  - Özellikle büyük mesajlar alınıp verilirken verimli ve kompakt bir IPC mekanizmasıdır.
  - Çift yönlü akış (bidirectional streaming) hem RPI hem mesajlaşma stilinde iletişimi mümkün kılar.
  - Çok farklı dillerde yazılmış istemciler ve servisler arasında birlikte çalışabilirlik sağlar.
- Eksiler:
  - JavaScript istemcilerinin gRPC tabanlı bir API'yi tüketmesi REST/JSON tabanlı API'lere göre daha fazla iş gerektirir.
  - Eski güvenlik duvarları HTTP/2'yi desteklemeyebilir.
  - Öğrencinin eklediği eksik: mesajlar insanın okuyamayacağı ikili formattadır.

gRPC, REST'e güçlü bir alternatiftir; ama REST gibi senkron bir iletişim mekanizması olduğu için o da kısmi hata (partial failure) sorununu yaşar.

7. Kısmi hata ve Circuit Breaker

7.1 Sorun

Öğrenciye göre REST'te de gRPC'de de yaşanan bu sorun genel olarak uzaktan çağrı problemidir. İstekler sürekli gelmeye devam eder ve cevap verilmezse bir trafik oluşur; bu trafik sonunda bütün kaynakları tüketir ve uygulama çalışamaz hale gelir.

Ekrandaki kitap şekli sorunu somutlaştırır (ekrandan, 00:41:20). Mobil uygulama `POST /orders` isteğini API gateway'deki "Create order endpoint"e gönderir. Gateway içindeki `OrderServiceProxy` isteği yine `POST /orders` ile Order Service'e iletir. Order Service cevap vermiyorsa, `OrderServiceProxy`'nin naif bir gerçekleştirmesi cevabı sonsuza kadar bekleyerek bloke olur. Bu hem kötü bir kullanıcı deneyimine yol açar hem de birçok uygulamada thread gibi değerli bir kaynağı tüketir. Sonunda API gateway'in kaynakları biter ve istekleri işleyemez hale gelir; bütün API kullanılamaz olur. Bu yüzden servisler, kısmi hataların uygulama boyunca yayılmasını önleyecek biçimde tasarlanmalıdır. Çözümün iki parçası vardır: `OrderServiceProxy` gibi RPI vekillerini cevap vermeyen uzak servislerle başa çıkacak biçimde tasarlamak ve çöken bir uzak servisten nasıl toparlanılacağına karar vermek.

7.2 Önlemler

Öğrencinin saydığı yaklaşımlar şunlardır:

- Ağ zaman aşımı (network timeout): her isteğe bir süre ayrılır; süre aşılınca istek iptal edilir.
- İstek sınırı: bir istemci belli bir sayıdan fazla istekte bulunamaz.
- Circuit Breaker: başarılı ve başarısız istek sayılarına göre bir değerlendirme yapılır; eşiğin aşıldığı durumda gelen istekler doğrudan reddedilir.

Slayttaki desen tanımı şöyledir (ekrandan, 00:41:20): Circuit Breaker, ardışık hata sayısı belirli bir eşiği aştıktan sonra bir zaman aşımı süresi boyunca çağrıları hemen reddeden bir RPI vekilidir.

> **Ders sonrası not:** Sözlü anlatımda Circuit Breaker "başarı/başarısızlık oranı eşiğin altındaysa o istemciden gelen istekleri reddeder" diye anlatıldı. Slayttaki tanıma göre reddedilen, istemcinin değil cevap vermeyen uzak servise yapılan çağrılardır;

tetikleyici de ardışık hata sayısının eşiği aşmasıdır ve red bir zaman aşımı süresi boyunca sürer.

7.3 Toparlanma

Tıkanan bir servisten toparlanmak için öğrenci iki yol söyler: ya istekler bırakılır (hata döndürülür) ya da önbellekteki eski sürümler (önceki cevaplar) döndürülür.

8. Servis keşfi

Bir istek yapabilmek için servisin ağdaki konumunu, yani IP adresini ve portunu bilmek gerekir. Günümüzde IP adresleri çoğunlukla dinamiktir ve bu bir sorun yaratır. Bu soruna çözüm olarak servis keşfi (service discovery) geliştirilmiştir. İki temel yaklaşım vardır: servis kayıt defteri (service registry) kullanmak ve dağıtım altyapısını (Docker, Kubernetes gibi araçlar) kullanmak.

Uygulama düzeyindeki servis keşfi deseninde servis kayıt defteri adlı bir uygulama ve bir API vardır. Servisler kendilerini bu kayıt defterine kaydettirir; bir istek geldiğinde kayıtlı ve boşta olan örneklerden biri isteği karşılar. Bu alanda yük dengeleme algoritmaları kullanılabilir.

Diğer yaklaşım platformun sağladığı keşiftir. Kubernetes ve Docker gibi ortamlarda keşif sanal IP aracılığıyla yapılır ve dağıtım tamamen bu ortamlar tarafından denetlenir. Artısı, keşfin tamamını platformun karşılaması ve bütün dillerde, bütün çerçevelerde çalışabilmesidir. Eksisi ise örneğin Kubernetes'in yalnız Kubernetes üzerindeki servislerde arama yapabilmesi ve yalnız onlar aracılığıyla çalışabilmesidir.

Hoca, bu sunumun birinci sunumdan sonra gelmesinin konu ve mantık akışı açısından daha iyi olduğunu belirtir.

9. Saga ile işlem koordinasyonu: neden gerekli

Üçüncü sunum, kitabın saga ile işlemleri koordine etme konusunu anlatır. Öğrenci önce sagaya neden ihtiyaç duyulduğunu göstermek için mikroservislere kısa bir giriş yapar.

9.1 Mikroservisler ve zorlukları

Mikroservis, bir uygulamayı birbirinden bağımsız çalışan, küçük ve birbirinden izole servisler olarak tasarlama yöntemidir. Bu servisler tek bir işlevi yerine getirmek üzere tasarlanır ve birlikte koordineli çalışarak büyük bir senaryonun parçalarını oluşturur. Mikroservis mimarisinin dayanıklılık, esneklik ve ölçeklenebilirlik, modülerlik, bağımsız kullanılabilirlik ve uyumluluk gibi pek çok avantajı vardır. Ancak bu avantajlar bazı zorlukları da beraberinde getirir. Bunlardan biri, servislere yayılmış dağıtık işlemler (distributed transaction) yönetilirken veri tutarsızlığının ortaya çıkmasıdır. Saga modelleri bunu önlemek için gerekir.

9.2 Para transferi örneği

Bir finansal hizmet uygulamasında para transferi yapılacağını ve bunun için iki mikroservis gerektiğini düşünelim: hesap mikroservisi ve transfer mikroservisi. Hesap mikroservisi müşterinin hesabından belirli bir miktarı düşmekten, transfer mikroservisi ise parayı alıcı bankanın hesabına yatırmaktan sorumludur.

Sistem monolitik olsaydı iki işlem de aynı veri tabanında çalışır ve tek bir transaction içinde atomik bir yapı oluşturulurdu. Mikroservislerde ise her mikroservisin kendi veri tabanı vardır ve bu işlemleri koordine etmek zorlaşır. Müşterinin hesabından para düşülür, ama transfer mikroservisinde teknik bir hata olur ve para alıcıya iletilmezse bir tutarsızlık doğar: süreç yarıda kalmıştır. Bu tutarsızlığı önlemek için iki yöntemden biri kullanılabilir: saga ya da dağıtık transaction. Örneğin sisteme paranın müşterinin hesabına geri yatırılmasını sağlayan bir telafi işlemi eklenirse bu iş saga ile yapılmış olur. Bu yapı mikroservislerin bağımsız çalışmasını korurken işlem tutarlılığı için ek bir koordinasyon mantığı sağlar.

9.3 ACID

Bu işlemler gerçekleşirken veri tutarlılığı da sağlanmalıdır. Slayta göre (ekrandan, 00:50:00) bir transaction'ın başarılı sayılabilmesi için ACID özelliklerini taşıması gerekir:

- Bütünlük (atomicity): transaction'lar ya tamamen başarıyla gerçekleşmeli ya da hiç gerçekleşmemelidir.
- Tutarlılık (consistency): transaction sonunda bütün veriler tutarlı olmalı, veriler arasında tutarsızlık olmamalıdır. Örneğin bir hesaptan para düşülüyor ve başka bir hesaba aktarılıyorsa, aktarılan hesapta da o kadar artış beklenir.
- Yalıtım (isolation): bir transaction başka bir transaction'dan etkilenmemelidir.
- Dayanıklılık (durability): slayttaki tanıma göre transaction sırasında hata olursa veriler önceki haline döndürülebilmelidir.

  Ders sonrası not: Slayttaki dayanıklılık tanımı hatalıdır. Dayanıklılık, onaylanmış (commit edilmiş) bir transaction'ın sonucunun sistem çökse bile kalıcı olması demektir. Hata durumunda önceki duruma geri dönmek bütünlüğün (atomicity) konusudur.

9.4 Saga tasarım deseni

Saga tasarım deseni, birden çok mikroservise yayılan transaction'ları yönetmek için kullanılır. Her adım bir önceki adımın başarılı olmasıyla tetiklenir. Bir hata tespit edildiğinde ya hata önlenir ya da önceki duruma dönülür; hata varken sürecin devam etmesi engellenir. Süreçte bir hata alınırsa daha önce işlenmiş transaction'lar geri alınır (rollback). Böylece bütünlük, yani atomik yapı tutarlı biçimde sağlanır.

Saga desenini uygulamanın iki yaklaşımı vardır: koreografi ve orkestrasyon. Koreografide karar alma ve işlemler katılımcılar arasında dağıtılmıştır. Orkestrasyonda ise ek olarak merkezi bir sınıf, orkestratör vardır; servisler arasındaki iletişim onun aracılığıyla kurulur ve yönetilir.

10. Koreografi tabanlı saga

10.1 Mantık

Koreografi tabanlı sagada sağanın her adımı bir olay (event) yayınlar, bu olay diğer servisler tarafından dinlenir ve süreç adım adım ilerler. Merkezi bir kontrol sistemi yoktur. Bu, dağıtık ve bağımsız çalışma sağlar; servislerin esnek bir yapısı vardır. Tasarım açısından olayların doğru sırada işlenmesi ve hata yönetimi kritiktir, çünkü dışarıdan bir denetleyici yoktur, süreç yalnız servisler arası iletişimle ilerler. Servisler arası bağımlılıklar ve veri tutarlılığı iyi yönetilmelidir. Bu modelle bir sistem yönetilecekse tasarım en başta iyi modellenmelidir; yoksa ileride daha büyük sorunlarla karşılaşılır.

Dezavantajları: servisler arası olay trafiği artar ve bu izlemeyi zorlaştırır; hata ve geri alma işlemlerinin yönetimi karmaşıklaşır; tutarlılık için ek bir mekanizmaya ihtiyaç duyulur.

10.2 E-ticaret örneği

Bu yaklaşımda merkezi bir yönetici yoktur; yalnız servisler ve aralarındaki event iletişimi vardır. Her servis işini tamamlayınca bir event fırlatır, sonraki servis bu event'i tüketip sürecine devam eder. Slayttaki akış şöyledir (ekrandan, 00:54:40):

1. `Order Service siparişi oluşturur, Pending statüsünde ORDER_CREATED_EVENT fırlatır.`
2. `Payment Service ORDER_CREATED_EVENT'i dinler, ödemeyi gerçekleştirir, BILLED_ORDER_EVENT fırlatır.`
3. `Stock Service BILLED_ORDER_EVENT'i dinler, stok güncellemesini yapar, ORDER_PREPARED_EVENT fırlatır.`
4. `Shipment (Delivery) Service ORDER_PREPARED_EVENT'i dinler, kargo işlemlerini yapar, ORDER_DELIVERED_EVENT fırlatır.`
5. `Order Service ORDER_DELIVERED_EVENT'i dinler ve siparişin durumunu tamamlandıya çeker.`

Arada hata yoksa süreç, servislerin event fırlatması ve dinlemesiyle tamamlanır.

10.3 Hata durumu

Akışta hata olursa geri alma (rollback) için yeni bir event fırlatılması gerekir. Öğrencinin gösterdiği küçük kesitte Stock Service bir `PRODUCT_OUT_OF_STOCK` event'i yayınlar (ürün stokta yok). Bu event'i Order Service ve Payment Service dinler. Payment Service ödemeyi geri almalı, Order Service de siparişin statüsünü başarısıza (fail) çekmelidir ki hata devam etmesin ve sistem önceki duruma dönsün.

10.4 Artılar ve eksiler

Koreografi sagası basit bir yapıya sahiptir; süreç servisler ve event'ler üzerinden ilerler. Yeni bir servis eklemek ve gerçekleştirmek basit senaryolarda kolaydır. Ancak karmaşık senaryolarda tasarımın önemi ortaya çıkar ve yapı karmaşıklaşır: hatalar ve gözden kaçabilecek durumlar artar, çünkü orkestrasyondaki gibi bir kontrol sistemi yoktur. Sistemi tam anlamıyla test etmek için bütün servislerin çalışır durumda olması gerekir; bu da testi zorlaştırır.

11. Orkestrasyon tabanlı saga

11.1 Mantık

Orkestrasyon sagası, sağa işlemini yönetmek için koreografiye ek olarak bir orkestratör sınıfı kullanır. Bu sınıfın tek sorumluluğu saga katılımcılarına ne yapacaklarını söylemektir. Çalışma biçimi komut ve geri dönüş (reply) etkileşimidir: orkestratör bir katılımcıya belirli bir işlemi yapması için bir komut mesajı gönderir; katılımcı işlemi bitirince orkestratöre bir geri dönüş mesajı göndererek işlemin tamamlandığını bildirir. Orkestratör geri dönüş mesajını işler ve süreci bir sonraki saga adımına aktarır.

En belirgin artısı, diğer yönteme göre merkezi bir kontrol yapısının olmasıdır. Orkestratör bütün süreç üzerinde tam denetim sağlar ve işlem sırasında yönetime açıkça katılır; bu da karmaşıklığı azaltır. Buna karşılık ek bir orkestratör sınıfı olduğu ve denetim daha sağlam bir yapıyla yürüdüğü için maliyet artar.

11.2 E-ticaret örneği

Slayttaki örnekte bir sipariş saga orkestratörü (Order Saga Orchestrator) vardır (ekrandan, 01:00:00). Bu orkestratör, abone olan bütün tüketicilere ne zaman ne yapacaklarını ileten bir tüketicidir. Akış şöyledir:

```
Order Service    : siparişi Pending durumunda oluşturur,
                   Orchestrator'a ORDER_CREATED bilgisini gönderir;
                   Orchestrator sipariş oluşturma transaction'ını başlatır.
Orchestrator     -> EXECUTE_PAYMENT (Payment Channel)  -> Payment Service
Payment Service  -> Payment Executed Reply
Orchestrator     -> UPDATE_STOCK (Order Channel)       -> Stock Service
Stock Service    -> Order Prepared Reply
Orchestrator     -> ORDER_DELIVER (Delivery Channel)   -> Shipment/Delivery Service
Delivery Service -> Order Delivered Reply
(bütün cevaplar Order Saga Reply Channel üzerinden orkestratöre döner;
 kanallar bir mesaj aracısı içindedir)
```

Slayttaki not: her durum için saga üzerinde durum (state) tutmak, sürecin hangi adımda yanlış yönetildiğini görmeyi kolaylaştırır.

11.3 Hata durumu

Orkestratör bir siparişin oluşturulması için gereken akışı bilir. Olası bir hata durumunda (fail order) işlemi geri almak için abone olan bütün servislere geri alma işlemini koordine etmekten sorumludur. Bir katılımcı başarısızlık bildirdiğinde orkestratör işlemin başarısız olduğu bilgisini alır ve rollback'i başlatır. Öğrenci rollback'i şöyle özetler: sagaya ihtiyaç duyulmasının nedeni, bir tutarsızlık ya da hatalı yapı oluştuğunda hata öncesi duruma geri dönebilmektir.

11.4 Artılar ve eksiler

Orkestrasyon sagasında transaction yönetimi merkezileşir; artık bir merkezi yönetici vardır. Gerçekleştirmesi ve test edilmesi diğer yönteme göre daha kolaydır, çünkü süreç servisler ve event'ler arasında değil, merkezi bir yönetici tarafından daha az adımda yürütülür. Daha basittir: yeni adımlar eklendiğinde işlem karmaşıklığı koreografiye göre daha az artar. Birbirini bekleyen işlemlerin olduğu bir senaryoda bu yöntemi kullanmak daha mantıklıdır. Eksisi altyapı karmaşıklığıdır.

12. Altıgen mimari ve iş mantığı yaklaşımları

Dördüncü sunum bu haftanın ilk sunumudur ve iş mantığını düzenleme desenlerini (business logic organization patterns) anlatır.

12.1 Altıgen mimari

Öğrenci önce iş mantığının merkezde bulunduğu altıgen (hexagonal) mimariyi anlatır (ekrandan, 01:05:30). Altıgen mimari, büyük ya da küçük bir uygulamayı çevresindeki bütün sistemlerden (kullanıcı arayüzü, veri tabanı vb.) bağımsızlaştırır. Merkezde iş mantığı bulunur. İş mantığı kullanıcı arayüzlerinden ve veri tabanlarından bağımsız olduğu için her parça kendi içinde ayrı test edilebilir.

İş mantığının çevresinde adaptörler bulunur:

- Gelen (inbound) adaptörler:
  - REST API: istemciden gelen HTTP isteklerini (`POST /orders`, `GET /order/Id`) alarak iş mantığını başlatır.
  - Order command handlers: sipariş servisine gelen komut isteklerini (Order Service requests) alır; örneğin `createOrder` metoduyla iş mantığındaki ilgili sınıfı tetikleyip komutun gerçekleşmesini sağlar.
- Giden (outbound) adaptörler:
  - Domain event publisher adapter: iş mantığının ürettiği önemli olayları (Order events) diğer servislere ya da bileşenlere aktarır.
  - Database adapter: veri tabanı işlemlerini soyutlar ve iş mantığından bağımsız hale getirir. Daha sonra anlatılacak repository taktiksel deseniyle veri ekleme, güncelleme, silme gibi işlemleri yapar (Order database).

12.2 Nesneye yönelik ve prosedürel yaklaşım

İş mantığını kurmanın iki yaklaşımı vardır. Sınıfları ve nesneleri gerçek dünya varlıklarıyla temsil etmek isteniyorsa nesneye yönelik bir iş mantığı kurulur. Her işlemi metotlarla, işlem adımlarıyla tanımlamak isteniyorsa prosedürel yaklaşım kullanılır. Hangisinin seçileceği uygulamanın büyüklüğüne ve karmaşıklığına bağlıdır. Buna karşılık gelen iki düzenleme deseni vardır: Transaction Script ve Domain Model.

13. Transaction Script deseni

Transaction Script deseninde kullanıcıdan gelen her istek ayrı bir prosedürle (transaction script) ele alınır. Nesneye yönelik bir yapı olmadığından iş akışı işlemlerden oluşur; her istek bir işlem yöntemiyle karşılanır.

Slayttaki yapı şöyledir (ekrandan, 01:08:20):

```
Davranış içeren sınıf (classes with behavior):
  OrderService
    createOrder()
    reviseOrder()
    cancelOrder()
    ...

Durum içeren sınıflar (classes with state):
  OrderDao              Order
```

```
save(Order)             orderId
findOrderById()         orderLineItems
...                     ...
```

İş mantığı genellikle bir hizmet sınıfında (service class), örneğin `OrderService` sınıfında yazılır; bu sınıfta her sistem işlemi için ayrı bir yöntem bulunur. Bütün işlemler bu ortak sınıftan yapılır, alan (domain) bazında ayrılmaz. İş mantığı veri tabanına veri erişim nesneleri (Data Access Objects, DAO) aracılığıyla erişir; örneğin `OrderService`, veri tabanı işlemleri için `OrderDao`'yu kullanır. Veri nesneleri (ör. `Order` sınıfı) yalnız veri depolamak için kullanılır, davranışları yoktur.

Bu yapı basit olduğu için basit uygulamalarda tercih edilir. Artıları: nesneye yönelimin karmaşıklığı olmadığından işlemler hızlıdır ve basittir; her işlem bir yönteme karşılık gelir, bu doğrudanlık takibi kolaylaştırır. Eksileri: iş mantığı büyüyüp karmaşıklaşırsa takip zorlaşır ve kod karmaşıklaşır, çünkü her şey tek bir servis sınıfından yönetilir; ayrıca kalıtım gibi nesneye yönelik programlama özelliklerinden yararlanılamaz.

Slayta göre ne zaman kullanılır (ekrandan, 01:10:00): iş mantığı basit ve işlem odaklıysa ve daha karmaşık bir nesneye yönelik model oluşturmak gereksizse. Transaction Script basit iş mantığını hızla ve etkin biçimde düzenlemek için ideal bir yöntemdir; iş mantığı karmaşıklaştığında Domain Model gibi daha güçlü tasarım desenlerine geçmek daha iyi bir tercihtir.

14. Domain Model deseni ve Domain-Driven Design

14.1 Domain Model deseni

Domain Model deseninde sınıflar artık yalnız davranıştan ya da yalnız durumdan oluşmaz; hem davranışı hem durumu (veriyi) tutar. Gerçek dünya kavramlarını temsil eden küçük sınıflardan oluştuğu için tasarımı daha iyidir. Sınıflar iş alanını (domain) temsil ettiği için kodun anlaşılabilirliği artar; hangi sınıfın hangi kavramla ilgili iş yaptığı ve rolü daha iyi bilinir, daha iyi düzenlenebilir.

Nasıl çalıştığına bakılırsa yine bir `OrderService` vardır, ama bu sınıf yalnız yönetim ve yönlendirme için kullanılır. `createOrder`, `reviseOrder`, `cancelOrder` gibi işlemler ayrı bir domain'de, `Order` domain sınıfında yapılır. Transaction Script'teki gibi bütün metotlar tek bir serviste toplanmaz, ayrı domain'lere dağıtılır. Yalnız durumdan oluşan, yalnız değer alan sınıflar da vardır: örneğin teslimat zamanı ve adresi gibi bilgileri tutan `DeliveryInformation`. Veri tabanı işlemleri repository deseniyle `OrderRepository` sınıfında yapılır. Bu yapı ayrı domain'lerden oluştuğu için daha düzenlidir; sürdürülebilirliği ve ileride kullanılabilirliği artar.

Artıları:

- Küçük sınıflardan oluştuğu ve her kod ayrı domain'lerde bulunduğu için anlaşılması ve güncellenmesi kolaydır; değişiklikte yalnız ilgili domain güncellenir.
- `Account`, `BankingTransaction` gibi sınıflar doğrudan gerçek dünyadaki kavramlara karşılık geldiği için ne yaptıkları kolayca anlaşılır.
- Her sınıf bağımsız olarak test edilebilir.
- Strategy ve Template Method gibi tasarım desenleriyle genişletilebilir; örneğin Template Method ile hazır bir kodun üzerinden genişletme yapılabilir.

Eksisi: domain model çok büyüdüğünde sınıflar diğer sınıflara bağımlı hale gelebilir. Bir domain sınıfı ihtiyaç duyduğu bilgiyi başka bir sınıfta arayabilir ve bu bağımsızlığı kaybettirir. Her domain'in bağımsız kalmasını sağlamak için Domain-Driven Design metodolojisi kullanılır.

14.2 Domain-Driven Design

Domain-Driven Design (DDD), işi bilen alan uzmanlarıyla (domain expert) yazılımcılar arasında iş birliği sağlamak ve ortak bir dil bulmak amacıyla kullanılan bir metodolojidir. DDD'de bütün uygulama için tek, ortak bir domain modeli yoktur; her servisin kendine ait bir domain modeli vardır. Ortak bir domain modeliyle yürütülen işlemlerde bağımsızlık sorunu ortaya çıkıyordu; her iş alanının kendi domain modeli olunca bu sorun ortadan kalkar.

Her domain modelin sınırları çizilir. Sınırlı bağlam (bounded context) terimiyle anlatılan bu yapıda her domain model kendi sınırları içinde, kendi kurallarıyla çalışır; her alt alan kendine özgü kurallar ve veri modeliyle tanımlanır.

DDD'de iki tür desen vardır:

- Stratejik desenler: alanı, her biri ayrı bir iş alanını temsil eden alt alanlara (subdomain) böler.
- Taktiksel desenler: domain modeldeki sınıfların rollerini ve özelliklerini düzenler:
  - Entity (varlık): benzersiz bir kimliği olan sınıf; ör. `orderId`'si olan `Order`.
  - Value object (değer nesnesi): benzersiz kimliği olmayan, yalnız değerlerden oluşan sınıf; ör. teslimatın adresini ve zamanını tutan teslimat bilgisi sınıfı.
  - Service: diğer sınıfları yöneten, iş mantığının başlatıldığı sınıf; ör. `OrderService`.
  - Factory: nesne oluşturmayı düzenler; ör. bir `OrderFactory` sınıfında statik bir `create` metodu.
  - Repository: veri tabanı işlemlerini (ekleme, silme, güncelleme) soyutlar.

DDD'nin artıları: önce stratejik desenlerle alt alanlara bölündüğü ve sınırlı bağlamlarla sınırlar çizildiği için uygulama daha iyi organize edilir (modülerlik); her domain model gerçek dünyadaki bir kavrama karşılık geldiği için anlaşılabilirlik artar; taktiksel desenlerle test edilebilirlik artar; kodun genişletilmesi kolaylaşır.

Sonuç olarak DDD, iş mantığı karmaşıklaştığında Domain Model ile birlikte kullanılması gereken bir metodolojidir. Stratejik desenler uygulamayı mantıksal alt alanlara böler; taktiksel desenler domain modeldeki sınıfları ve bileşenleri düzenler. DDD'nin sınırlı bağlam ve aggregate gibi kavramları servisleri daha iyi ve bağımsız hale getirir. Öğrenci aggregate kavramına, bir sonraki konu olduğu için girmez.

15. Domain event'ler: tanım ve kullanım nedenleri

Beşinci sunum, bir önceki sunumda anlatılan DDD'nin bir parçası olan domain event'lerin yayınlanmasını anlatır. Öğrenci konuyu anlamayı kolaylaştırmak için yer yer kod incelemesine başvurur.

15.1 Tanım

Slayta göre Merriam-Webster sözlüğündeki event tanımları şunlardır (ekrandan, 01:20:00): bir şeyin olması; dikkate değer bir durumun meydana gelmesi; bir sosyal etkinlik ya da faaliyet; tıbbi açıdan olumsuz bir olay. Konuyla en ilişkili olan ikinci tanımdır. Öğrenci event'i bir durumdan diğerine geçiş, yani durumun (state) değişmesi olarak da tanımlar. DDD bağlamında bir domain event, iş alanında meydana gelen önemli bir durumu temsil eder.

Sipariş sisteminden örnekler: Sipariş Oluşturuldu (Order Created), Sipariş İptal Edildi (Order Cancelled), Sipariş Gönderildi (Order Shipped). Event'lere geçmiş zaman fiiliyle ad verilmesine dikkat edilmelidir.

Domain event'ler genellikle aggregate'lerle ilişkilidir. Aggregate'ler, DDD'de tutarlılığı ve değişimi yöneten ana yapı taşlarıdır. Domain Event deseni, bir aggregate oluşturulduğunda ya da önemli bir durum değişikliğine uğradığında bir domain event yayınlanmasını öngörür. Bu, iş alanındaki değişiklikleri izlemek ve iletişim kurmak açısından önemlidir; ölçeklenebilirlik ve denetlenebilirlik sağlar, sistem bileşenlerinin birbirine bağımlılığını azaltır.

Öğrenci kısa bir örnekte `createOrder` adlı bir metodun domain event'leri bir liste olarak tuttuğunu ve bir event handler'ın bunları işlediğini gösterir; ayrıntıları sonraki kısımlarda inceler.

15.2 Neden domain event kullanılır

Domain event'ler aggregate'lerin durum değişikliklerini diğer taraflara iletmekte güçlü ve etkilidir. Öğrencinin saydığı kullanım nedenleri şunlardır:

- Son kullanıcının davranış analizi: bir ürünü beğenmek ya da sepete eklemek bir event'tir (ör. "sepete eklendi"). Hangi üründe ne kadar gezildiği, bir event'in ne kadar çağrıldığı

üzerinden iş analizleri yapılır; kullanıcıya göre ürün önermek ya da iş süreçlerini geliştirmek mümkün olur. Öğrenci bunun günümüzde event'lerle mi yoksa yeni teknolojilerle mi yapıldığının değişmiş olabileceğini, ama hâlâ böyle çalışan sistemler olduğunu belirtir.
- Domain event'lerin takibiyle sistem doğruluğunun izlenmesi: sistem beklenmedik bir durumla karşılaştığında hata event'ler üzerinden tespit edilebilir. Geri bildirim göndermek en yaygın kullanım nedenlerinden biridir. Örnek: ATM'de tutar girilip onaylandığında bir event gönderilir; bu event tutulur ve kullanıcıya "paranız sayılıyor/hazırlanıyor" gibi bir bildirim gösterilir.
- Diğer iş süreçlerinin tetiklenmesi: bir ödeme onaylandığında sipariş hazırlama süreci başlar; ödeme onayı event'i başka bir sürecin tetikleyicisi olur.
- Sistem içindeki bileşenlerin tetiklenmesi: örneğin bir ayar değiştiğinde dosyaya yazma metodunun çağrılması.

Bu sayede uygulama mimarisinde iş süreçlerini optimize etme, veri tutarlılığını koruma ve kullanıcı deneyimini geliştirme gibi avantajlar elde edilir.

16. Event sınıfı, meta veri ve event zenginleştirme

16.1 Event sınıfının içeriği

Event adı genellikle geçmiş zaman fiiliyle oluşturulur. Bir domain event sınıfı iki tür bilgi içerir:

- Anlamlı özellikler: event'in taşıdığı bilgiyi ifade eden temel özellikler.
- Meta veriler: event ID'si ve zaman damgası (timestamp) gibi ek bilgiler. Zaman damgası denetim ve izleme süreçlerini kolaylaştırır. Olayı tetikleyen kullanıcının kimliği de meta veri olabilir; ama bilinmesi gerekmediği durumlarda yer almayabilir.

Meta veriler iki yaklaşımla iletilebilir. Birincisinde meta veri doğrudan event nesnesinin (event object) içindedir, domain event'in bir parçasıdır. İkincisinde meta veriler bir zarf (envelope) nesnesiyle sarılır. Örneğin `DomainEventEnvelope` sınıfı zaman damgasını, ID'yi ve event nesnesinin kendisini birlikte tutar. Bu sınıf generic olduğu için herhangi bir türdeki domain event'i sarmalayabilir.

16.2 Event zenginleştirme

Event zenginleştirme (event enrichment), event nesnesinin tüketiciye gerekli bütün bilgiyi sağlayacak biçimde ayrıntılandırılmasıdır. Bu, tüketicinin servisten ek bilgi istemesi ihtiyacını ortadan kaldırır.

Sunumun başındaki "sipariş oluşturuldu" (`OrderCreated`) örneğinde event siparişin oluşturulduğunu bildirir ve temel bilgileri içerir. Ama event'in tüketicisi (öğrencinin örneğinde restoran tarafındaki servis) siparişi işlemek için daha fazla bilgiye ihtiyaç duyabilir. İki çözüm vardır:

- Tüketici bu bilgileri event'in kaynağından sorgular. Ama bu, her seferinde bir sorgu göndermeyi gerektirir ve sistemin performansını yavaşlatır.
- Tüketicinin ihtiyaç duyduğu bilgiler event'in içine konur. Öğrencinin örneğinde `OrderEvent`'i gerçekleştiren bir `OrderCreated` sınıfı tüketicinin ihtiyaç duyduğu bütün bilgileri taşır. Bu, ilk yöntemin sorununu önler; ama bakım maliyetini artırır, çünkü gereksinimler değiştikçe event sınıfının içeriği de değiştirilmelidir.

17. Domain event'leri belirleme ve event storming

17.1 Gereksinimlerden yola çıkma

Domain event'leri belirlemenin en temel yolu gereksinimlerden yola çıkmaktır (ekrandan, 01:30:00). Gereksinim analizinde genellikle "X olduğunda Y'yi yap" biçiminde ifadeler kullanılır. Bu ifadeler bir bildirim ya da tepki mekanizmasının varlığını, dolayısıyla bir domain event ihtiyacını gösterir. Slayttaki örnek:

- Gereksinim: "Yemek kuryeye teslim edildiğinde tüketiciye yola çıktı bildiriminin gönderilmesi".
- Domain event: `DagitimaCikti`.

17.2 Event storming

İkinci yöntem event storming'dir. Daha popülerdir ve günümüz iş süreçlerine daha uygundur. Büyük sistemler için kullanılır; küçük sistemlerde gereksinim analizi daha hızlı çözüme götürebilir. Büyük sistemlerde alan uzmanları bir araya getirilir ve bir beyin fırtınası ortamı yaratılır. Büyük bir beyaz tahta ya da masa ve renkli yapışkan notlar (post-it) kullanılır. Event'ler, komutlar ve aggregate'ler hızlıca modellenir. Zaman akışı kabaca oluşturulur; amaç ayrıntılı analizden çok sistemin akışını göstermektir. Adımlar şöyledir:

1. Event'ler üzerine beyin fırtınası: alan uzmanları sistemin hangi event'lere ihtiyaç duyabileceğini tartışır. Event'ler genellikle geçmiş zaman cümlesiyle turuncu post-itlere yazılır ve kaba bir zaman çizelgesine göre tahtaya yapıştırılır.
2. Event tetikleyicilerinin belirlenmesi: "Bu event bir kullanıcı eylemiyle mi tetikleniyor?" gibi sorular sorulur. Komutlar mavi post-itlerle, komutu veren aktör (kullanıcı) sarı post-itle, kullanılan arayüz (view/UI) yeşil post-itle gösterilir. Öğrencinin gösterdiği örnekte "sipariş ver" ve "çağrıyı sonlandır" komutları, sipariş arayüzü ve "sipariş analiz edildi", "e-posta gönderildi", "çağrı sonlandırıldı" gibi event'ler yer alır. Dış bir sistemin tetiklenmesi mor, katılımcıların bilgi sahibi olmadığı ya da ayrı ve karmaşık olan başka bir sistem (ör. yalnız "tedarikçi sistemi" diye geçilen sistem) pembe post-itlerle gösterilir.
3. Aggregate'lerin belirlenmesi: her komutu işleyen ve karşılık gelen event'i üreten aggregate'ler belirlenir ve sarı notlarla gösterilir.

Birkaç saatlik bir toplantıyla sistemin genel bir modeli ortaya çıkar.

17.3 Otopark örneği

Öğrencinin ikinci örneği bir otopark sistemidir (ekrandan, 01:34:45). Lejantta Command, View, Events, Aggregate, Actor ve External System renkleri gösterilir. Sistem üç aggregate'e bölünmüştür: Ticket machine, Parking ve Payment.

```
Ticket machine:
  [Actor: User] + [View: Ticket machine UI] -> [Command: Create new Ticket]
      -> [Event: TicketCreated]

Parking:
  [Actor: User] + [View: Online portal] -> [Command: Extend parking (online portal)]
      -> [Event: Parking Session Renewed]
  [External: System] -> [Command: Expire Carpark Session]
      -> [Event: Parking session expired] -> [Command: Notify Customer]
      -> [Event: Customer Notified]

Payment:
  [Actor: User] + [External: Online portal (Stripe)] -> [Command: Pay for parking]
      -> [Event: Payment Successful] -> [Process payment]
      -> [Event: Parking Sessions Paid]
```

Bu, basit bir sistemin bu yöntemle nasıl modellendiğini gösterir.

18. Domain event'leri üretme, güvenilir yayınlama ve tüketme

18.1 Üretme ve yayınlama

Domain event'leri aggregate'ler yayınlar. Öğrenci bunu bir kütüphaneden kitap ödünç alma sistemiyle açıklar: burada aggregate kitabın kendisidir, çünkü durumu (ör. ödünç alınma durumu) değişen odur. Aggregate durumunun ne zaman değiştiğini bilir ve buna göre hangi event'in yayınlanacağına karar verir.

Ancak aggregate'lerin doğrudan bir mesajlaşma API'sini çağırması uygun değildir, çünkü bu altyapıyı iş mantığıyla iç içe geçirir. Bunun yerine sorumluluk aggregate ile servis arasında bölünür:

- Aggregate durum değişikliğinde event'leri üretir ve servise döndürür.
- Servis bu event'leri alır, mesajlaşma API'sine erişir ve event'leri yayınlar.

Öğrencinin gösterdiği kod örneğinde bir `Ticket` aggregate sınıfı, durum değişikliği sırasında ürettiği event'leri bir liste olarak tutar ve dönüş değeri olarak döndürür (`accept` metodu). Örneğin devamında Kitchen Service, `DomainEventPublisher`'ı çağırarak bu event'leri yayınlar. Temel mantık budur: aggregate event'i döndürür, servis de yayınlayıcının metodunu çağırarak event'i yayınlar.

18.2 Güvenilir yayınlama

Event'lerin güvenilir biçimde yayınlanması önemlidir, çünkü konu veri tabanı işlemleriyle ilişkilidir (ekrandan, 01:38:30). Bir event yayınlandığında veri tabanındaki işlem gerçekleşmemiş olabilir ya da tersine veri tabanı güncellendiği halde event yayınlanmamış olabilir. Event aggregate durumu güncellenmeden yayınlanırsa sistem yanlış bir duruma girebilir; event yayınlandıktan sonra aggregate durumu güncellenemezse diğer sistemler gerçek durumu yansıtmayan bilgilerle çalışır. Bu yüzden event'ler yerel bir veri tabanı işleminin parçası olarak gönderilmelidir; bu, event'lerin yalnız aggregate durumu başarıyla güncellendiğinde mesajlaşma sistemiyle paylaşıldığını garanti eder.

Transactional messaging ile güvenilir yayın, aggregate'in güncellenmesini ve event'lerin yayınlanmasını tek bir ACID işleminde birleştirir. Böylece iki işlem ya birlikte başarılı olur ya da başarısızlıkta tamamen geri alınır. Bu yaklaşımda:

- Domain event'ler veri tabanında OUTBOX adlı özel bir tabloya kaydedilir.
- Veri tabanı işlemi tamamlandıktan sonra event'ler bir mesaj aracısına (message broker) gönderilir.
- Eventuate Tram çerçevesi bu mekanizmayı uygular ve event'leri veri tabanı işlemleriyle birlikte yönetir.

  Ek bilgi (derste anlatılmadı): Bu yaklaşım literatürde "Transactional Outbox" deseni diye bilinir. OUTBOX tablosundaki kayıtları mesaj aracısına taşıyan ayrı bir bileşen (message relay) vardır; bu bileşen tabloyu periyodik olarak sorgulayabilir (polling publisher) ya da veri tabanının işlem günlüğünü izleyebilir (transaction log tailing).

18.3 Tüketme

Event'lerin ele alınmasını kolaylaştırmak için çerçeve bir `DomainEventDispatcher` mekanizması sunar. Bu mekanizma mesaj aracısından gelen event'leri uygun handler'lara yönlendirir. Öğrencinin gösterdiği şemada Kitchen Service'in event tüketicisi (`KitchenServiceEventConsumer`) ve domain event handler'ları yer alır; en sonda restoran menüsünün güncel kalmasını sağlayan bir işlev (`reviseMenu`) çağrılır.

Öğrencinin vardığı sonuç (ekrandan, 01:40:00): domain event'ler iş mantığını modellemek ve servisler arası iletişimi sağlamak için güçlü bir araçtır; event odaklı mimariler ölçeklenebilir ve esnek sistemler kurmak için uygun bir zemin hazırlar; doğru tasarlanmış bir domain event sistemi karmaşık iş süreçlerini düzenlemeye ve sistem tutarlılığını artırmaya olanak tanır.

19. Örnek sistem: kitap ödünç alma

Öğrenci, anlatılanları çok sadeleştirilmiş bir kitap ödünç alma sistemi üzerinden kodla özetler. Önce `KitapOduncAlindiEvent` sınıfı tanımlanır (ekrandan, 01:41:00). Bu sınıf, kitap ödünç alma işlemine dair bütün bilgileri içeren bir domain event'tir ve `DomainEvent` arayüzünü gerçekleştirir; yani bir olay olduğu bu arayüzle belirtilir. Özellikleri kitap kimliği, kullanıcı kimliği ve alış tarihidir; bir kurucu ve get metotları vardır:

```
public class KitapOduncAlindiEvent implements DomainEvent {
    private final String kitapId;
    private final String kullaniciId;
    private final ZonedDateTime alisTarihi;

    public KitapOduncAlindiEvent(String kitapId, String kullaniciId, ZonedDateTime alisTarihi)
        this.kitapId = kitapId;
        this.kullaniciId = kullaniciId;
```

```
        this.alisTarihi = alisTarihi;
    }

    public String getKitapId() {
        return kitapId;
    }

    public String getKullaniciId() {
        return kullaniciId;
    }

    public ZonedDateTime getAlisTarihi() {
        return alisTarihi;
    }
}
```

Bu kod derste slaytta gösterilen bir parçadır; derlenmemiştir.

Ardından aggregate gelir. Burada aggregate kitabın kendisidir. Kitabın ödünç alma metodu (`oduncAl`) çağrıldığında durumu değişir ve tetiklenen domain event bir listeye eklenir. Bu event'ler dönüş değeri olarak dışarıya, yani servise gönderilir. Aggregate ile servisin ayrılması dediğimiz şey budur.

Son olarak bir kitap servis sınıfı vardır. Bu sınıf veri tabanına erişim kısımlarını içerir ve yayınlayıcıyı (publisher) tanımlar. Servis, kitabın kimliği gibi bilgileri alır, kitabın ödünç alma metodunu çağırır ve dönen event'leri yayınlar. Öğrenci son olarak `DomainEventPublisher` arayüzünün içeriğini ve yayınlanan event'in nasıl tutulup sisteme ya da kullanıcıya gösterileceğini basit bir sınıfla gösterir.

20. Saga türlerinin karşılaştırması ve event sourcing

Altıncı ve son sunum, event sourcing ile sagaların birlikte kullanımını anlatır. Öğrenci önce saga türlerini ve event sourcing'i kısaca tanıtır.

20.1 Saga türleri

Saga, monolitik bir yapı yerine işlerin servis bazında ayrıldığı, her servisin kendi işlemlerini yönettiği ve kendi işlemi bitince bir event yayınlayarak sonraki servisi tetiklediği bir yapıdır. İki çeşidi vardır.

Koreografi tabanlı sagalarda merkezi bir yönetim noktası yoktur; servisler kendi aralarında domain event'ler üzerinden haberleşir.

- Artılar: basit akışlar için uygundur; ek bir servis gerektirmez (orkestrasyondaki gibi bir orkestratör servisi yoktur); sistemin sorumluluğu servislere dağıtıldığı için tek hata noktası (single point of failure) yoktur, yani bir servis çöktüğünde bütün sistemin çöktüğü bir nokta yoktur.
- Eksiler: her servis kendi event'lerini yönetir ve diğer servisler bu event'lere bağımlıdır; yeni servis eklendikçe hangi servisin hangisiyle haberleştiğini takip etmek zorlaşır ve sistemin karmaşıklığı artar. Döngüsel bağımlılık (circular dependency) riski vardır: bir servisin bağımlı olduğu servis de başka bir servise, o da ilkine bağımlı olabilir. Bir işlemi simüle etmek, yani sistemi baştan sona test etmek için bütün servislerin ayakta olması gerekir.

Orkestratör tabanlı sagalarda her servisin kendini yönetmesi yerine bütün sistem tek bir orkestratör tarafından yönetilir.

- Artılar: çok katılımcılı ya da zamanla yeni katılımcı eklenebilecek karmaşık iş akışları için idealdir. Servisler birbirleriyle değil yalnız orkestratörle haberleşir: servis cevabını orkestratöre döner, orkestratör de sonraki tetiklenecek servise haber verir. Bu, döngüsel bağımlılık riskini ortadan kaldırır. Her servisin işi yalnız event'i işleyip yanıt dönmek olduğu için yönetilebilirlik artar.

- Eksiler: kurgulamak için önce orkestratör kısmının ayrıntılı tasarlanması gerekir. Orkestratör bütün servisleri yönettiği için tek hata noktası olur: orkestratör çökerse bütün sistem çöker.

20.2 Event sourcing

Event sourcing, veriyi doğrudan değiştirmek yerine veri üzerinde gerçekleşen bütün değişikliklerin tutulduğu bir sistemdir. Yapılan işlemler kaydedilir. Örneğin bir sipariş güncellendiğinde sipariş doğrudan güncellenmez, "sipariş güncellendi" event'i kaydedilir. Bu kayıtların tutulduğu yere olay deposu (event store) denir.

Veri, verinin kendisi yerine üzerinde yapılan event'ler olarak saklansa da servisler arası veri bütünlüğünü korumak için sagalara katılmak gerekir. Örneğin bir sipariş servisi siparişi oluşturduğunda müşteri servisinin de güncellenmesi gerekir. Bu servisler event sourcing kullansalar bile genel sistemin yönetilebilmesi için sagalara katılmalıdır. Öğrenciye göre kitap bu konuyu dört alt başlıkta toplar: koreografi tabanlı sagaları uygulamak, orkestrasyon tabanlı saga oluşturmak, event sourcing tabanlı bir saga katılımcısı eklemek ve saga orkestratörünü event sourcing ile uygulamak.

21. Event sourcing ile koreografi sagaları

Event sourcing'in temel mantığı ile koreografi tabanlı sagaların temel mantığı birbirine çok paraleldir; bu yüzden ikisini birlikte kullanmak oldukça kolaydır. Bir aggregate güncellendiğinde bir event yayınlanır; başka bir aggregate'in event handler'ı bu event'i tüketir ve kendi aggregate'ini günceller. Öğrencinin gösterdiği örnekte bir servisin yayınladığı `CreditReserved` event'i, sipariş tarafındaki bir event handler tarafından tüketilir.

Slayta göre ikisinin birlikte iyi çalışmasının nedeni, event sourcing kullanımının sagaların ihtiyaç duyduğu özellikleri içermesidir (ekrandan, 01:50:00):

- Mesajlaşma tabanlı IPC: olaylar, servisler arasında iletişim kurmak için bir mesajlaşma mekanizması sağlar.
- Mesaj tekrarını önleme: event handler'ların idempotent yapısı sayesinde aynı mesajın birden fazla kez işlenmesi ve olası tutarsızlıklar önlenir. Idempotent, aynı işlemin ilk sonucu değiştirmeden birden çok kez tekrar işlenebilmesi anlamına gelir. Öğrencinin ifadesiyle: aynı event birden fazla kez yayınlanmışsa ya yalnız bir kez işlendiği garanti edilir ya da birden fazla işlense bile sonraki tekrarlar asıl sonucu değiştirmez.
- Durumun atomik güncellenmesi: event sourcing bir aggregate'i (slaytta "küme") güncellerken atomikliği garanti eder; böylece veri tutarlılığı korunur.

Ancak bir dezavantajı vardır: event'ler iki amaca hizmet eder. Event sourcing'de event'ler veri üzerindeki değişiklikleri saklar. Sagada ise bu değişiklikleri diğer servislere bildirmek gerekir; dahası veri değişmese bile bazı durumlar bildirilmelidir. Örneğin bir iş kuralı ihlal edildiğinde, bu bir hata olduğu için event sourcing açısından bir event oluşmaz; ama sagada diğer katılımcılara iş kuralının ihlal edildiğini bildiren bir yanıt mesajı gitmesi gerekir.

22. Event sourcing ile orkestrasyon sagaları ve katılımcıları

22.1 Orkestrasyon tabanlı saga oluşturma

Servisin `createOrder()` gibi metotları iki şey yapar: bir aggregate oluşturur ya da günceller ve bir saga orkestratörü oluşturur. Servis bu iki eylemi, birincisi yapılırsa ikincisinin de sonunda yapılacağını garanti edecek biçimde gerçekleştirmelidir. Yani burada bir transaction yapısından söz edilir: ilk eylem başarısız olursa ikincisi de geri alınmalı ya da hiç yapılmamalıdır.

Bunun nasıl garanti edileceği servisin kullandığı event store türüne bağlıdır. Event store ilişkisel bir veri tabanıysa iş kolaydır: ilişkisel veri tabanlarının transaction yönetimi bir hata durumunda doğrudan rollback desteği sağladığı için iki eylemin birlikte tamamlanması garanti edilebilir.

İlişkisel olmayan (NoSQL) bir event store kullanıldığında ise event store'u atomik olarak güncelleyip bir saga orkestratörü oluşturmak mümkün değildir, çünkü NoSQL veri tabanlarının transaction yapıları sınırlıdır. Bu durumda servis, yayınlanan domain event'lere cevap

olarak saga orkestratörünü oluşturan event handler'lar kullanmalıdır. Örneğin `OrderCreated` event'ini işleyen handler, sipariş oluşturma sagasını (`CreateOrderSaga`) başlatır. Burada aynı event'in tekrar tekrar işlenip birden çok saga oluşturulmaması garanti edilmelidir. Kitabın önerisi: event'e benzersiz (unique) bir ID vermek ve event handler'ın oluşturacağı sagada bu ID'yi kullanarak tekrarlanan mesajları engellemek.

22.2 Event sourcing tabanlı saga katılımcısı eklemek

Bir orkestrasyon sagasına katılması gereken yeni bir servis düşünelim. Servis ilişkisel bir event store kullanıyorsa, yukarıdaki nedenlerle mesajları atomik işlemek ve yanıt göndermek kolaydır. İlişkisel olmayan bir veri tabanı kullanıyorsa ya da aynı transaction'ın yönetilebileceği tek bir veri tabanı yoksa iki sorun çözülmelidir:

- Komut mesajlarının idempotent işlenmesi: servis tekrarlanan mesajları algılayıp atmalıdır. Çözüm, mesajı işlerken oluşan event'lere mesajın kimliğini kaydetmektir.
- Yanıt mesajlarının atomik gönderilmesi.

  Ders sonrası not: Öğrenci idempotence'ı bir yerde "bir mesajın en az bir kez işleneceğinin garantisi" diye tanımladı. Doğrusu: en az bir kez (at-least-once) teslim, mesajlaşma altyapısının verdiği garantidir; idempotence ise aynı mesaj birden çok kez işlense de sonucun değişmemesidir. Mesajlar en az bir kez teslim edildiği için idempotent işleme gerekir.

Öğrenci atomik yanıt göndermek için önce bir yaklaşımdan söz eder; bu yaklaşımın ne olduğu altyazıda okunamıyor. Bu yaklaşım iki sorun doğurur: bir saga komutu aggregate'in durumunu her zaman değiştirmeyebilir (hata durumunda durum değişmez, dolayısıyla event de oluşmaz) ve bu yaklaşımda saga orkestratörünün event sourcing tabanlı katılımcılara diğerlerinden farklı davranması gerekir.

Kitabın önerdiği daha iyi yaklaşım (ekrandan, 01:55:50): saga katılımcısı, saga orkestratörünün yanıt kanalına yanıt mesajı göndermeye devam eder; ama bunu doğrudan değil, iki adımlı bir işlemle yapar:

1. Bir saga event handler'ı bir aggregate oluşturduğunda ya da güncellediğinde, event store'a aggregate'in yayınladığı gerçek olaylarla birlikte sahte bir `SagaReplyRequested` event'inin kaydedilmesini sağlar.
2. `SagaReplyRequested` sahte olayı için bir event handler, olaydaki verileri kullanarak yanıt mesajını oluşturur ve bunu saga orkestratörünün yanıt kanalına yazar.

Öğrencinin şemadaki örneği şöyledir:

```
Saga orkestratörü -> authorize komutu -> Account komut kanalı
Saga command dispatcher -> Accounting Service komut handler'ı -> Account aggregate
Event store'a iki event yazılır:
  AccountAuthorized        (gerçek event)
  SagaReplyRequested       (sahte event)
SagaReplyRequested event handler'ı -> yanıt mesajını oluşturur
  -> saga orkestratörünün yanıt kanalına yazar
```

22.3 Saga orkestratörünü event sourcing ile uygulamak

Bir saga orkestratörü oluştururken yanıtlanması gereken üç soru vardır:

- Saga orkestratörü nasıl saklanır (persist edilir)? Öğrenci kitaptaki "persist" kelimesini "sürdürmek" diye çevirir.
- Orkestratörün durumu atomik olarak nasıl değiştirilir ve komut mesajları nasıl gönderilir?
- Orkestratörün yanıt mesajlarını tam bir kez işleyeceği, yani idempotent bir yapı kurulacağı nasıl garanti edilir?

Birinci sorunun cevabı: saga orkestratörünün çok basit bir yaşam döngüsü vardır. Önce oluşturulur, sonra saga katılımcılarından gelen yanıtlara göre güncellenir. Bu yüzden `SagaOrchestratorCreated` ve `SagaOrchestratorUpdated` event'leri, orkestratörün durumunu yeniden oluşturmak için gereken verileri tutar.

İkinci sorunun cevabı: orkestratörün göndermek istediği her komut için bir `SagaCommandEvent` kaydedilir. Bu event, komutu göndermek için gereken bütün bilgiyi, yani hedef kanalı ve komut nesnesini içerir; event store'a kaydedilir ve ardından bir event handler komutu ilgili kanala yönlendirir. Bu yaklaşım komutun en az bir kez gönderileceğini garanti eder. En az bir kez olduğu için aynı event birden fazla kez işlenebilir ya da aynı komut birden fazla kez yazılabilir riski vardır.

Üçüncü sorunun cevabı, önceki kısımlarda anlatılan yöntemdir: event'in benzersiz bir kimliği olmalı, bu kimlik bir yerde saklanmalı ve aynı kimlikle yeni bir event geldiğinde atlanmalıdır.

Dersin Toparlaması

Bu derste altı öğrenci sunumu, mikroservis kitabının üçüncü bölümden altıncı bölüme uzanan bir kesitini işledi. İlk iki sunum servislerin nasıl haberleştiğini anlattı: etkileşim stilleri, API-first tasarım, anlamsal sürümleme ve geriye uyumluluk, metin ve ikili mesaj formatları; ardından senkron RPI olarak REST ve gRPC, bunların ortak sorunu olan kısmi hata, buna karşı zaman aşımı, istek sınırı ve Circuit Breaker, son olarak da servis keşfi. Üçüncü sunum, servislerin ayrı veri tabanları yüzünden tek ACID transaction kullanılamadığı için işlemlerin saga ile, koreografi ya da orkestrasyon yoluyla nasıl koordine edildiğini e-ticaret örneğiyle gösterdi.

Son üç sunum servislerin içine baktı. İş mantığı altıgen mimarinin merkezine yerleştirildi ve Transaction Script ile Domain Model desenleri karşılaştırıldı; karmaşık iş mantığı için DDD'nin sınırlı bağlamları ve taktiksel desenleri önerildi. Domain event'ler aggregate'lerin durum değişikliklerini dışarıya taşıyan araç olarak tanıtıldı; gereksinimlerden ve event storming ile nasıl bulunacakları, outbox ile nasıl güvenilir yayınlanacakları anlatıldı. Son sunum event sourcing'in sagalarla birleşimini ele aldı: idempotent işleme, atomik yanıt için `SagaReplyRequested` sahte event'i ve orkestratörün `SagaOrchestratorCreated`, `SagaOrchestratorUpdated`, `SagaCommandEvent` event'leriyle saklanması.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.